

Document protocole

1. Introduction technique

Le protocole de communication est implémenté selon une architecture client-serveur via des communications par sockets TCP. Ici, TCP est privilégié pour garantir que les données soient bien transmises dans l'ordre et qu'aucun paquet ne soit perdu, ce qui est crucial pour la synchronisation en temps réel.

2. Structure des paquets

Chaque paquets envoyés suivra la structure suivantes:

- en-tête (header) :
 - ID du message
 - taille des données
- données (payload) :
 - ID du layer
 - ...

3. Tableau des messages

Authentification

MESSAGE	SENS	ARGUMENTS	DESCRIPTION
<code>AUTH_LOGIN_REQ</code>	client → serveur	<code>string pseudo</code> <code>string password</code>	Demande de se connecter
<code>AUTH_REGISTER_REQ</code>	client → serveur	<code>string pseudo</code> <code>string password</code>	Demande de s'enregistrer
<code>AUTH_RESULT</code>	serveur → client	<code>int succes</code>	Autorise/refuse la connexion/enregistrement

Lobby

MESSAGE	SENS	ARGUMENTS	DESCRIPTION
<code>LOB_INFO_USER_REQ</code>	client → serveur	<code>string pseudo</code> <code>string password</code>	Demande les informations du profil
<code>LOB_INFO_USER_REP</code>	serveur → client	<code>string pseudo</code> <code>string password</code>	Envoi les informations du profil
<code>LOB_CREATE_PROJECT_REQ</code>	client → serveur	<code>string role</code> <code>float size</code> <code>float scale</code>	Créer un projet

LOB_PROJECT_DATA_REQ	client → serveur	int PROJECT_ID int num_users std::vector users_name	Demande les données d'un projet (pour envoyer les couches, on utilise les messages adéquats)
LOB_PROJECT_DATA_REP	serveur → client	int PROJECT_ID int num_users std::vector users_name layer	Envoi les données d'un projet
LOB_SHARE_PROJECT_REQ	client → serveur	string user_role string shared_mode	Demande un code de partage
LOB_SHARE_PROJECT_REP	serveur → client	int code	Envoi un code de partage
LOB_JOIN_PROJECT_REQ	client → serveur	string pseudo	Demande de rejoindre un projet
LOB_EXPORT_PNG_PROJECT_REQ	client → serveur	int project_idx	Demande la transformation de la zone de base du projet dans le format souhaité
LOB_EXPORT_INT_OBJ_LAYER_REQ	client → serveur	int layer_idx int project_idx string sprite_path uint32 sprite_size uint8[] raw_png_data	Demande tout les objets se trouvant sur la couche en fonction de l'indice du projet
LOB_EXPORT_INT_OBJ_LAYER_REP	client → serveur	int layer_idx int project_idx string sprite_path uint32 sprite_size uint8[] raw_png_data	Demande tout les objets se trouvant sur la couche en fonction de l'indice du projet
LOB_EXPORT_INT_OBJ_LAYER_REQ	client → serveur	int layer_idx int project_idx string sprite_path uint32 sprite_size uint8[] raw_png_data	Demande tout les objets se trouvant sur la couche en fonction de l'indice du projet
LOB_EXPORT_INT_OBJ_LAYER_REP	serveur → client	int layer_depth	Envoie tout les objets se trouvant sur la couche
LOB_EXPORT_INT_PIX_LAYER_COORD_REQ	client → serveur	int x int y	Demande coordonnées de la couche
LOB_EXPORT_INT_PIX_LAYER_COORD_REP	serveur → client	int x int y	Envoie coordonnées de la couche
LOB_IMPORT_PROJECT_REQ	client → serveur	float size float scale	Créer un projet vide avec la taille et l'échelle. Ensuite update les couches en fonction de la lecture du fichier JSON lu depuis un fichier de sauvegarde (LOB_UPLOAD_OBJ_LAYER, LOB_UPLOAD_PIX_LAYER)
LOB_IMPORT_PROJECT_REP	serveur → client	bool succes int new_project_id	Renvoie la confirmation de la création d'un projet avec son id
LOB_OBJECT_LAYER_REQ	client → serveur	int num_objects std::vector	Demande couche objet

		<code><sf::renderTexture></code> <code>Object</code>	
<code>LOB_OBJECT_LAYER_REP</code>	serveur → client	<code>int num_objects</code> <code>std::vector</code> <code><sf::renderTexture></code> <code>Object</code>	Envoie couche objet
<code>LOB_PIXEL_LAYER_REP</code>	client → serveur	<code>sf::Image Layer</code>	Demande couche Pixel
<code>LOB_PIXEL_LAYER_REP</code>	serveur → client	<code>sf::Image Layer</code>	Envoie couche Pixel
<code>LOB_UPLOAD_OBJ_LAYER_REQ</code>	client → serveur	<code>int project_id</code> <code>int layer_index</code> <code>std::vector <Object></code> <code>list_objects</code>	Le client envoie la liste de tous les objets en fonction des couches qu'il a trouvé dans le JSON
<code>LOB_UPLOAD_OBJ_LAYER_REP</code>	serveur → client	<code>bool success</code>	Confirme la mise à jour des couches du projet
<code>LOB_UPLOAD_PIX_LAYER_REQ</code>	client → serveur	<code>int layer_index</code> <code>int project_id</code> <code>uint8[] raw_png_data</code>	Le client envoie au serveur la couche objet (png) sous forme de tableau d'octet pour le stocker.
<code>LOB_UPLOAD_PIX_LAYER_REP</code>	serveur → client	<code>bool success</code>	confirmation que la sauvegarde à été faite
<code>LOB_LIST_PROJECT_REQ</code>	client → serveur	<code>int num_proj</code> <code>std::vector <string></code> <code>name</code>	demande la liste des projets
<code>LOB_LIST_PROJECT_REP</code>	serveur → client	<code>int num_proj</code> <code>std::vector <string></code> <code>name</code>	envoi la liste des projets
<code>LOB_QUIT_PROJECT_REQ</code>	client → serveur	<code>int project_id</code>	informe le server que l'on retourne au lobby pour ne plus recevoir les données du projets

Carte

MESSAGE	SENS	ARGUMENTS	DESCRIPTION
<code>MAP_CREATE_LAYER_REQ</code>	client → serveur	<code>str type</code> <code>int layer_depth</code>	Créer une couche
<code>MAP_CREATE_LAYER_REP</code>	serveur → client	<code>str type</code> <code>int layer_depth</code>	Ajoute à tout les utilisateurs la nouvelle couche
<code>MAP_RENAME_LAYER_REQ</code>	client → serveur	<code>int layer_depth</code> <code>str name</code>	Renomme une couche
<code>MAP_RENAME_LAYER_REP</code>	serveur → client	<code>int layer_depth</code> <code>str name</code>	Informe tout les utilisateurs d'un nouveau nom pour une couches
<code>MAP_REMOVE_LAYER_REQ</code>	client → serveur	<code>int layer_depth</code>	Retire une couche
<code>MAP_REMOVE_LAYER_REP</code>	serveur → client	<code>int layer_depth</code>	Informe tout les utilisateurs de la suppression d'une couche
<code>MAP_LOCK_LAYER_REQ</code>	client → serveur	<code>int layer_depth</code> <code>User user</code>	Verrouille une couche

MAP_LOCK_LAYER_REP	serveur → client	int layer_depth User user	Informe tout les utilisateurs du verrouillage d'une couche
MAP_MOV_LAYER_REQ	client → serveur	int layer_depth int x int y	Déplace une couche
MAP_MOV_LAYER_REP	serveur → client	int layer_depth int x int y	Actualise la position d'une couche chez tout les utilisateurs
MAP_ORGANIZE_LAYER_REQ	client → serveur	int old_depth int new_depth	Déplace une couche dans la pile des couches
MAP_ORGANIZE_LAYER_REP	serveur → client	int old_depth int new_depth	Informe tout les utilisateurs du déplacement d'une couches dans la pile des couches
MAP_MASK_LAYER_REQ	client → serveur	int layer_depth visible bool	Modifie l'état de masquage une couche
MAP_MASK_LAYER_REP	serveur → client	int layer_depth visible bool	Modifie l'état masquage de la couche chez tout les autres utilisateurs
MAP_IMPORT_SPRITE_REQ	client → serveur	str file_name uint32 file_size uint8[] file	Importe un nouveau sprite utilisable (si le fichier est trop gros il faut le fragmenter)
MAP_IMPORT_SPRITE_REP	serveur → client	str path uint32 file_size uint8[] file	Ajoute a tout les utilisateurs le nouveau sprite importé
MAP_PUT_SPRITE_REQ	client → serveur	str path int layer_depth int x int y	Place un sprite
MAP_PUT_SPRITE_REP	serveur → client	str sprite_name int layer_depth int x int y	Place le sprite chez tout les utilisateurs
MAP_REMOVE_SPRITE_REQ	client → serveur	int layer_depth int x int y	Retire un sprite
MAP_REMOVE_SPRITE_REP	serveur → client	int layer_depth int x int y	Retire le sprite chez tout les utilisateurs
MAP_MOV_SPRITE_REQ	client → serveur	int layer_depth int x int y int new_x int new_y	Déplace un sprite
MAP_MOV_SPRITE_REP	serveur → client	int layer_depth int x int y int new_x int new_y	Actualise la position du sprite chez tout les utilisateurs
MAP_ROTATE_SPRITE_REQ	client → serveur	int layer_depth int x int y int angle	Tourne le sprite
MAP_ROTATE_SPRITE_REP	serveur → client	int layer_depth int x int y int angle	Actualise la rotation du sprite chez tout les utilisateurs

MAP_RESIZE_SPRITE_REQ	client → serveur	int layer_depth int x int y float multiplier	Redimensionne le sprite
MAP_RESIZE_SPRITE_REP	serveur → client	int layer_depth int x int y float multiplier	Redimensionne le sprite chez tout les utilisateurs
MAP_PUT_PIXEL_REQ	client → serveur	int layer_depth int x int y float size Shape shape Unit8 red Unit8 green Unit8 blue Unit8 opacity	Utilisation du brush
MAP_PUT_PIXEL_REP	serveur → client	int layer_depth int x int y float size Shape shape Unit8 red Unit8 green Unit8 blue Unit8 opacity	Utilisation du brush chez tout les autres utilisateurs
MAP_REMOVE_PIXEL_REQ	client → serveur	int layer_depth int x int y float size Shape shape	Utilisation de la gomme
MAP_REMOVE_PIXEL_REP	serveur → client	int layer_depth int x int y float size Shape shape	Utilisation de la gomme chez tout les autres utilisateurs

Chat

MESSAGE	SENS	ARGUMENTS	DESCRIPTION
CHAT_MESSAGE_REQ	client → serveur	string auteur string message	Envoyer un message
CHAT_MESSAGE_REP	serveur → client	string auteur string message	Envoyer le message a tout les utilisateurs
CHAT_CONNECT_USER_REP	serveur → client	string user	Informe tout les utilisateurs de la connexion d'un participant
CHAT_DISCONNECT_USER_REP	serveur → client	string user	Informe tout les utilisateurs de la déconnexion d'un participant
CHAT_LOCK_LAYER_REP	serveur → client	string auteur int depth	Informe tout les utilisateur du verrouillage d'une couche par un participant